Machine Learning and Deep Learning
Lecture-04

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Outline

- Principles of supervised machine learning
- K-Nearest Neighbors
- Linear Regression
- Logistic Regression
- L1/L2 Regularization
- Elastic Net

Supervised Learning → Choice of model (KNN, Regression) → Risk of overfitting → Regularization (L1, L2, Elastic Net) → Cross-validation for reliable evaluation

Supervised Learning

Fundamentals of Machine Learning

- Supervised Learning:
  - Predicting values.
  - Known targets.
  - User inputs correct answers to learn from.
  - Machine uses the information to guess new answers.
- Categories:
  - Regression: Estimate continuous values.
  - Classification: Identify a unique class.

How supervised ML works?

- Collect data and preprocess it
- Split a dataset into training and test data
- Choose an algorithm and set its hyperparameters.
- Train a model with the training data
- Evaluate how well the model works with the test data
- Repeat steps 2-4 until you are satisfied with the results

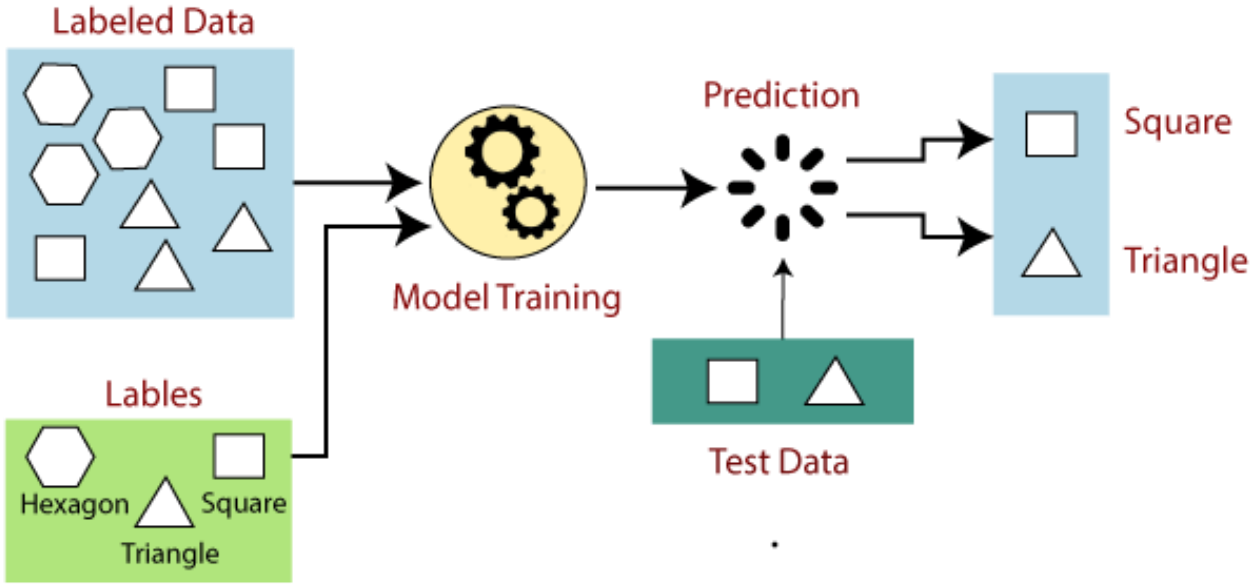
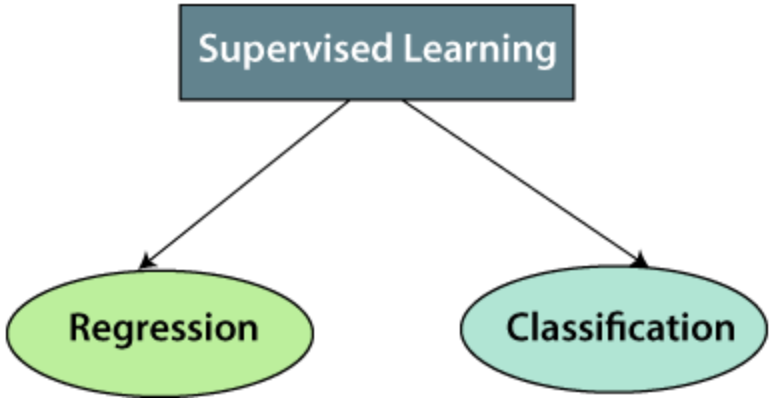


Image src: https://www.javatpoint.com/supervised-machine-learning

Supervised Categories



(Real-valued output)

(Discrete values, Boolean Categories)

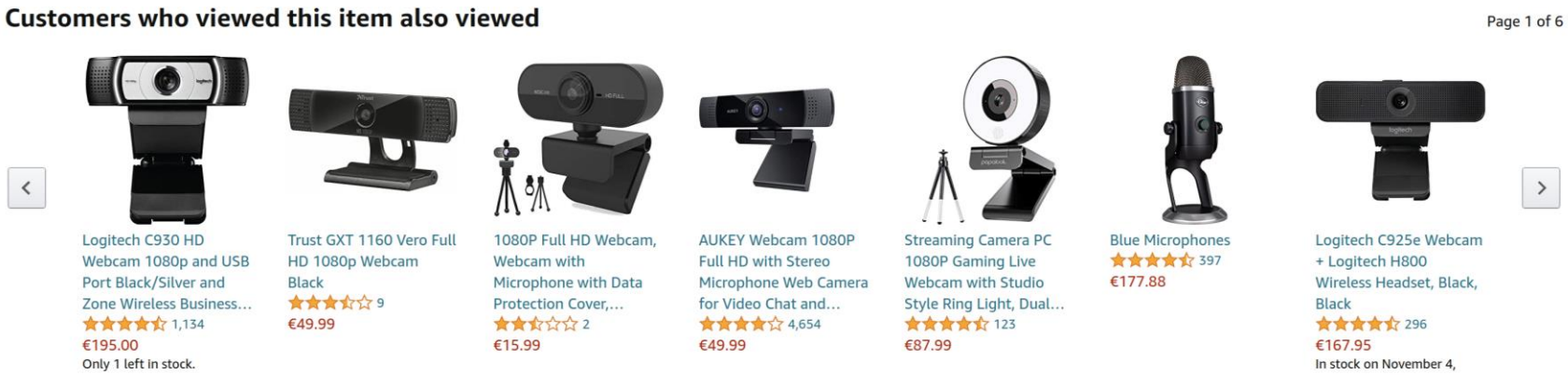
- Linear Regression
- Neural Networks*

Some NNs can be unsupervised, such as autoencoders and restricted Boltzmann.

- Decision Trees,
- K-Nearest Neighbors
- Support Vector Machine
- Logistic Regression
- Random Forests

Three C's of ML

- Three C’s of ML:
  - Collaborative filtering
  - Clustering
  - Classification
- Collaborative filtering is a technique for recommendations
  - It’s one primary type of recommender system
  - Can use the same algorithm to recommend practically anything – Movies (Netflix)
  - Amazon uses CF to recommend a variety of products



Three C's of ML

- Classification is a form of ‘supervised’ learning.
  - Requires training with data that has known labels.
  - Learns how to label new records based on that information.

- Applications:
  - Spam filtering: Train using a set of spam and non/spam messages --> System will eventually learn to detect unwanted e-mail.

  - Risk Analysis: Train using financial records of customers who do/don’t default --> System will eventually learn to identify risk customers

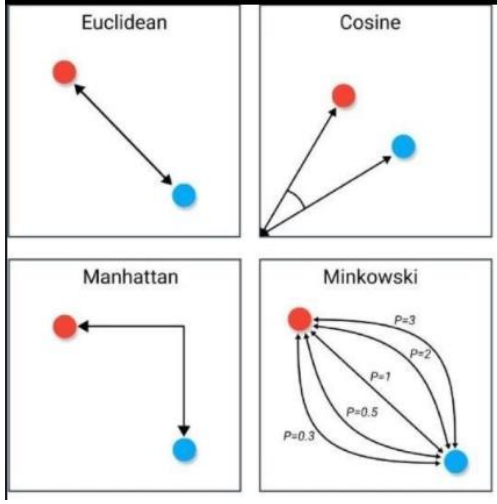
K-Nearest Neighbors: KNN

K-Nearest Neighbors: KNN

- Non-parametric, supervised learning classifier.
- Uses proximity to make classifications or predictions about grouping of an individual data point.
- Can be used for either regression or classification but used for classification.
- Evelyn Fix and Joseph Hodges are credited with the initial ideas around the KNN model in 1951.
- Application: Commonly used for simple recommendation systems, pattern recognition, data mining, financial market predictions, intrusion detection.
- Issues: As dataset grows, KNN becomes increasingly inefficient, compromising overall model performance. [Scaling problem]

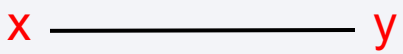
https://www.ibm.com/in-en/topics/knn

Compute KNN: Distance Metrics



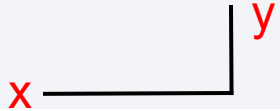
- Goal: Identify nearest neighbors of a given query point, so that we can assign a class label to that point.
- Determine your distance metrics using:

$$d(x,y) = \sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}$$



- Popular distance measure
- Limited to real-valued vectors.

$$\text{Manhattan Distance} = d(x,y) = \left(\sum_{i=1}^{m} |x_i - y_i|\right)$$



Measures absolute value between two points.

$$\text{Minkowski Distance} = \left(\sum_{i=1}^{n} |x_i - y_i|\right)^{1/p}$$

Generalized form of Euclidean and Manhattan distance metrics.

Compute KNN: Defining k

- K-value in k-NN algorithm defines how many neighbors will be checked to determine the classification of a specific query point.
  - Defining k is important but not easy.
- Different k-values can lead to overfitting or underfitting.
  - Lower values of k can have high variance, but low bias.
  - Larger values of k may lead to high bias and lower variance.

https://www.ibm.com/in-en/topics/knn

Compute KNN: Defining k

- Choice of k will largely depend on input data.
- Data with more outliers or noise will likely perform better with higher values of k.
- Must have an odd number for k to avoid ties in classification.
- Cross-validation can help choose optimal k for your dataset.

https://www.ibm.com/in-en/topics/knn

K-nearest neighbors

- Given a training dataset X and a new instance x_{new}
- Find k points in X that are closest to x_{new}
- Using the selected distance measure:
- Predict label for x_{new}
- Classification: majority vote among the k nearest neighbors.
- Regression: mean of the k nearest neighbors.

sklearn.neighbors.KNeighborsClassifier

K-nearest neighbors

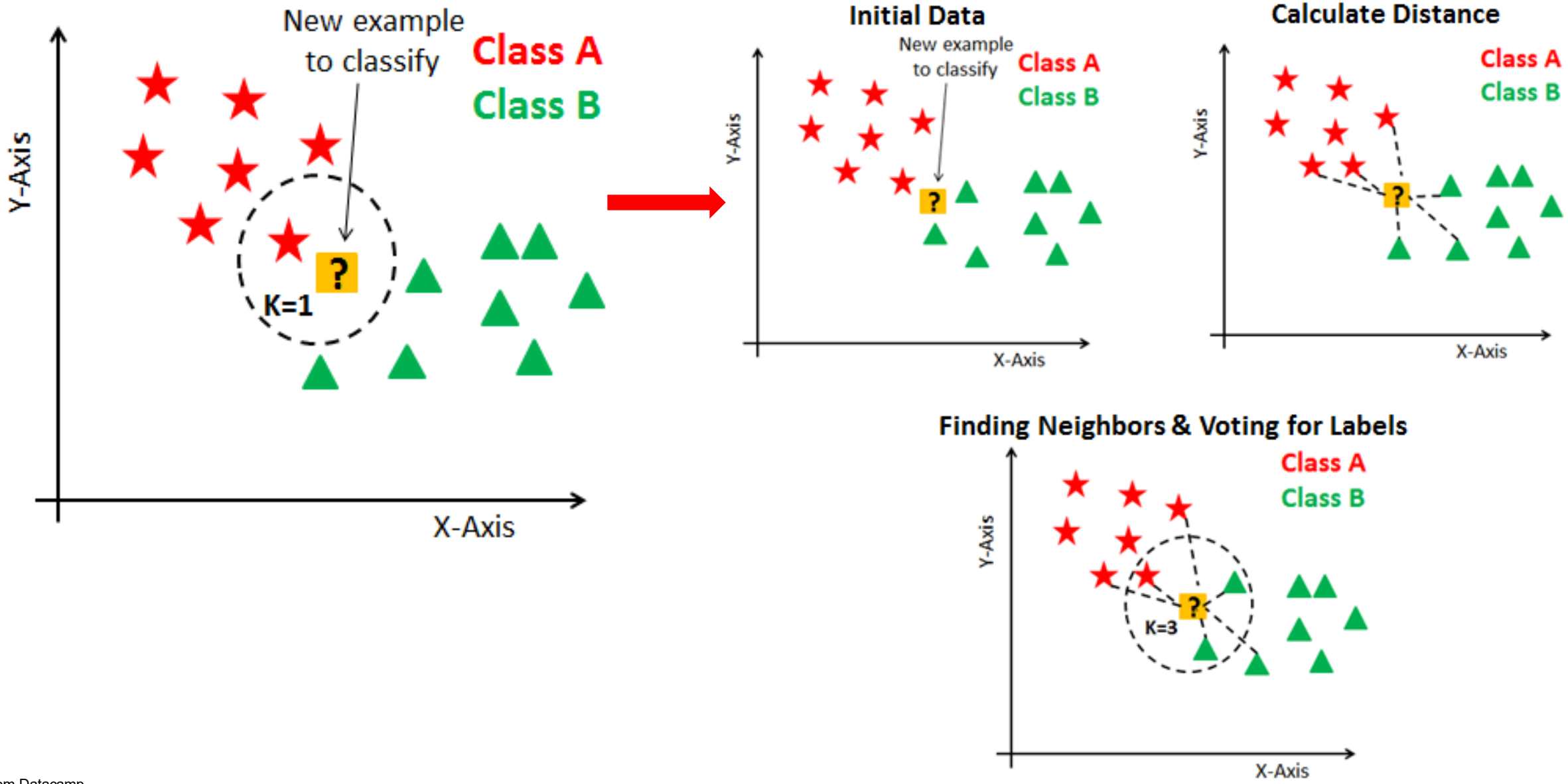


Image from Datacamp

Compute KNN: Code

```
from sklearn.neighbors import KNeighborsClassifier
model_name = 'K-Nearest Neighbor Classifier'
knnClassifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p=2)
knn_model = Pipeline(steps=[('preprocessor', preprocessorForFeatures), ('classifier' ,
knnClassifier)])
knn_model.fit(X_train, y_train)
y_pred = knn_model.predict(X_test)
```

Applications of k-NN

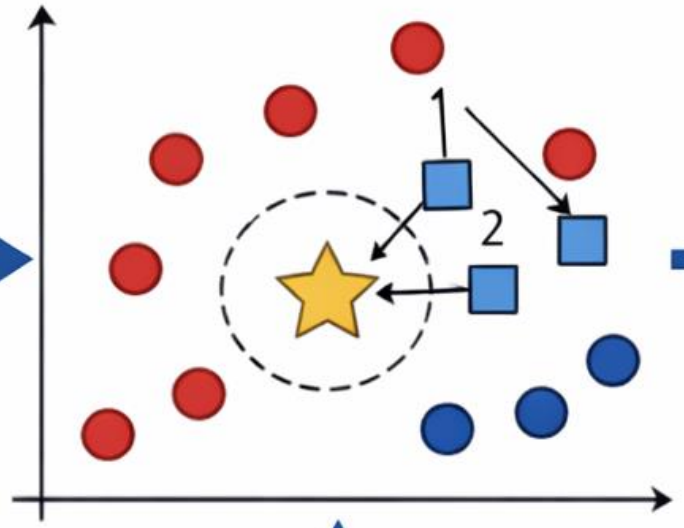
1. Data preprocessing: Datasets frequently have missing values, but KNN can estimate for those values [Missing data imputation].
2. Recommendation Engines: Using clickstream data from websites, KNN can provide automatic recommendations to users on additional content.
3. Finance: Using KNN on credit data can help banks assess risk of a loan.
4. Healthcare: Predicting risk of heart attacks and prostate cancer.
5. Pattern Recognition: KNN has also assisted in identifying patterns in text and in digit classification.

https://www.ibm.com/in-en/topics/knn

Missing data imputation

Incomplete Data

Feature A	Feature B	Feature C
5.2	2.1	?
4.8	1.9	6.3
5.0	2.3	?
4.6	2.0	7.1
5.1	?	6.5



Imputed Data

Feature A	Feature B	Feature C
5.2	2.1	6.2*
4.8	1.9	6.3
5.0	2.3	6.5*
4.6	2.0	7.1
5.1	2.2*	6.5

Find Nearest Neighbors

Impute Missing Value
(e.g., Average of Neighbors)

Advantages and Disadvantages

- Advantages:
  - Easy to implement
  - Adapts easily
  - Few hyperparameters

- Disadvantages:
  - Does not scale well
  - Curse of dimensionality
  - Prone to overfitting

Remember!!

- Features need to be scaled:
    - A feature with a big scale can dominate all the distances.
    - A feature with a small scale would get neglected.

- “Curse of dimensionality”:
    - Problems with high-dimensional spaces (e.g. more than 5 features).
    - Volume of space grows exponentially with dimension.
    - Need more points to ‘fill’ a high-dimensional volume.

Linear Regression

Linear Regression

- Linear Regression is a supervised learning algorithm used for regression, not classification.
- Linear regression analysis is used to predict the value of a variable based on the value of another variable.
  - Dependent variable: You want to predict.
  - Independent variable: You are using to predict the other variable's value.
- Estimates the coefficients of the linear equation, involving one or more independent variables that best predict the value of the dependent variable.

Linear Regression

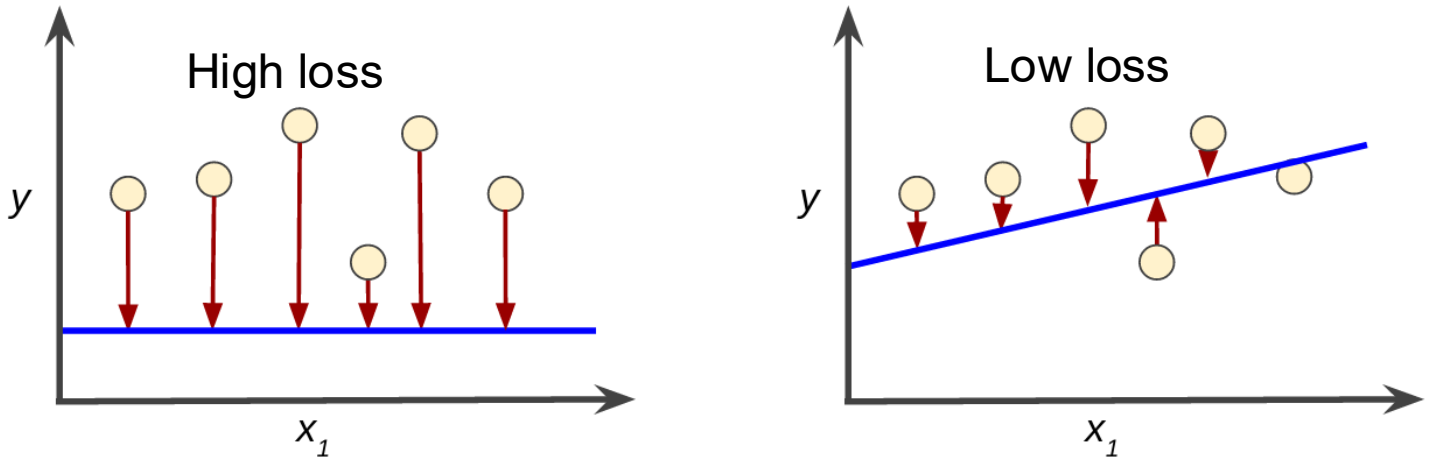
- You can take large amounts of raw data and transform it into actionable information.
  - Better insights by uncovering patterns and relationships.

- For example, performing an analysis of sales and purchase data can help you uncover specific purchasing patterns on particular days or at certain times.

- Disadvantages:
  - Performs poorly when there are non-linear relationships.
  - Sensitive to outliers.

Linear Regression

- A common method for regression problems.
- Linear regression models are parametric models.
- Fits a straight line or surface that minimizes the discrepancies between predicted and actual output values.

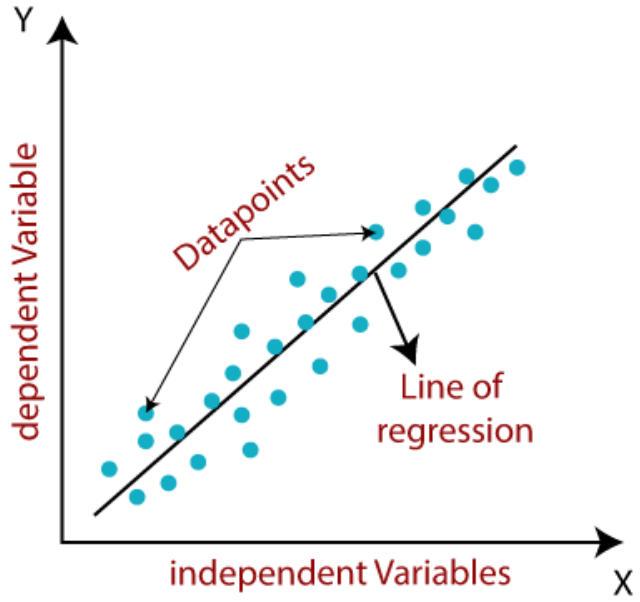


- Simple linear regression calculators use a “least squares” method to discover the best-fit line for a set of paired data.

https://developers.google.com/machine-learning/crash-course/descending-into-ml/training-and-loss

Linear Regression

- In statistics, a simple linear regression is represented as:
- $y = mx + b$



- ***y*** *is* the value we're trying to predict
- ***m*** is the slope of the line
- ***x*** is the value of our input feature
- ***b*** is the y-intercept

- Linear-regression models are relatively simple.
- Linear regression is a long-established statistical procedure.
- Linear-regression models are well understood and can be trained very quickly.

Linear Regression

- LR: $y' = b + w_1 x_1$; $y' = b + w_1x_1 + w_2x_2 + w_3x_3$ (Three features)
  - $\boldsymbol{y'}$ is predicted label (a desired output)
  - $\boldsymbol{b}$ is bias (y-intercept)
  - w_1 is weight of feature 1 (slope);
  - $\boldsymbol{x_1}$ is feature (input)

sklearn.linear_model.LinearRegression()

- ML algorithm for linear regression attempts to find values for b and w_i that minimizes the loss (e.g. mean squared error).

Linear Regression: Key Assumptions

- Data: Dependent and independent variables should be quantitative. (Convert Categorical variables to binary).
  - For each variable: Consider number of valid cases, mean and SD.
- For each model: Consider regression coefficients, correlation matrix, variance-covariance matrix etc
- Plots: Consider scatterplots, partial plots, histograms.
- Other assumptions:
  - For each value of the independent variable, the distribution of the dependent variable must be normal.
  - The relationship between the dependent variable and each independent variable should be linear and all observations should be independent.

https://www.ibm.com/topics/linear-regression

Linear Regression: Data Assumptions

- Variables should be measured at a continuous level (time, sales, weight).
- Use a scatterplot to find out quickly if there is a linear relationship between those two variables.
- The observations should be independent of each other.
- Your data should have no significant outliers.
- Check that the variances along the best-fit linear-regression line remain similar all through that line.
- The residuals (errors) of the best-fit regression line follow normal distribution.

https://www.ibm.com/topics/linear-regression

Linear Regression

- Linear regression models identify relationship between a continuous dependent variable and one or more independent variables.
- Linear regression produces a probability.
- Simple linear regression supports one independent variable and one dependent variable
- Multiple linear regression supports more independent variables.
- For each type of linear regression, it seeks to plot a line of best fit through a set of data points.

Logistic Regression

Logistic Regression

- Logistic Regression is a supervised learning algorithm used for classification.
- Logistic regression is a data analysis technique that uses mathematics to find the relationships between two data factors.
  - Then uses relationship to predict the value of one of those factors based on the other.
  - Prediction usually has a finite number of outcomes, like yes or no.
- Steps:
  - Identify the question: Frame the question to get particular outcomes:
    - Do rainy days impact our monthly sales? (yes or no)
    - What type of credit card activity is the customer performing? (authorized, fraudulent, or potentially fraudulent)
  - Collect historical data
  - Train the regression analysis model
  - Make predictions for unknown values
- It cannot predict actual values for continuous data.
  - It can answer questions like "Will the price of rice increase by 50% in 10 years?"

Logistic Regression

- Logistic regression estimates the probability of an event occurring.
  - Example: voted or didn’t vote, based on a given dataset of independent variables.
  - Outcome is a probability, dependent variable is bounded between 0 and 1
- Also known as logit model.
- Often used for classification and predictive analytics.
- A logit transformation is applied on the P(Success)/ P(Failure). [log odds, or the natural logarithm of odds].
- Logistic regression does require large sample size.

Logistic Regression Equation

$$\text{Logit}(pi) = 1/(1+ \exp(-pi))$$

$$\ln(pi/(1-pi)) = Beta_0 + Beta_1*X_1 + \ldots + B_k*K_k$$

- logit(pi) is the dependent variable and x is the independent variable.
- Beta parameter is estimated via maximum likelihood estimation (MLE).
- Process:
  1. MLE tests different values of beta through multiple iterations to optimize for the best fit of log odds.
  2. Iterations produce the log likelihood function, and logistic regression seeks to maximize this function to find the best parameter estimate.
  3. Once the optimal coefficient is found, the conditional probabilities for each observation can be calculated, logged, and summed together to yield a predicted probability.

- For binary classification, a probability < 0.5 will predict 0 while a probability > 0 will predict 1.
- Evaluate "How well the model predicts the dependent variable?"[goodness of fit].

https://www.ibm.com/topics/logistic-regression

Logistic Regression

- Logistic regression needs an adequate sample to represent values across all the response categories.
- Three types of logistic regression models, which are defined based on categorical response.
  - Binary logistic regression
  - Multinomial logistic regression
  - Ordinal logistic regression

Logistic Regression Types

- Binary logistic regression: Dependent variable has only two possible outcomes (e.g. 0 or 1).
  - Ex: Predicting if an e-mail is spam or not spam.
  - Most common classifiers for binary classification.

- Multinomial logistic regression: Dependent variable has three or more possible outcomes but no specified order.
  - Example: Movie studios want to predict what genre of film a movie goer is likely to see, to market films more effectively.

- Ordinal logistic regression: Dependent variable has three or more possible outcomes but with specified order.
  - Example: Grading scales from 0 to 12.

Logistic Regression and ML

- Logistic regression attempts to distinguish between classes (or categories).
- It cannot generate information [such as image] of the class that it is trying to predict (e.g. a picture of a cat) [Discriminative model].
- Within ML, the negative log likelihood used as the loss function, using the process of gradient descent to find the global maximum.
- Logistic regression can also be prone to overfitting, when there is a high number of predictor variables within the model.
- Regularization is typically used to penalize parameters large coefficients when the model suffers from high dimensionality.

https://www.ibm.com/topics/logistic-regression

Logistic Regression

- It is used to make a prediction about a categorical variable versus a continuous one.
- A categorical variable can be true/false, yes/no, 1/0.
- Logit function transforms the S-curve into straight line.
- Use cases:
  - Fraud detection: Can help to identify data anomalies, which are predictive of fraud.
  - Disease prediction: Can be used to predict the likelihood of disease.
  - Churn prediction: Specific behaviors may be indicative of churn in different functions of an organization.

Penalized Approach: Regularization

Context

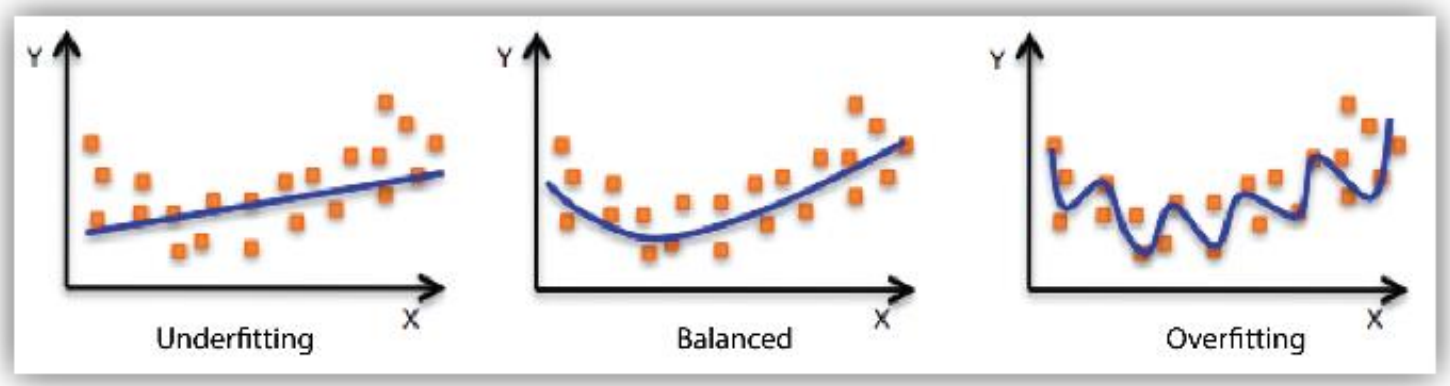
A high-dimensional problem is one in which the number of variables ***d*** is much larger than the number of observations ***n***.

1. High-dimensional problems: genetics, neauroscience
2. Usual least square estimator does not work!!

Regularization

Overfitting occurs when a model achieves near-perfect accuracy on training data but fails to generalize to new, unseen data.

- "True" mapping from x_i to y_i is complex need a large combination of features (high-degree polynomial).
- Need to know which features are relevant (otherwise models tend to overfit).
  - Solution: Regularization: constrain model by adding a penalty term to the loss function based on model complexity.
  - L2-regularization penalizes sum of the squares of the coefficients
  - L2 discourages large weights.



Regression:
Use overfitting and underfitting

Img: https://docs.aws.amazon.com/machine-learning/latest/dg/model-fit-underfitting-vs-overfitting.html

L2-Regularization

- Goal: Balance getting low training error vs. having small weights.
  - Large slopes (w_j) allow the model to react "violently" to small changes in input.
  - Small Slopes force the model to be smoother and less sensitive to outliers.
- Regularization modifies the objective function (minimizes it)

$$\text{Cost} = \text{Sum of Squared Errors} + \lambda \sum_{j=1}^{n} w_j^2$$

- The hyperparameter λ > 0 controls the "strength" of the constraint:
  - **Small λ:** Fits data perfectly (Risk: **Overfitting**).
  - **Large λ:** Keeps weights small (Risk: **Underfitting**).
  - **Optimal λ:** Found via cross-validation to minimize error on *unseen* data.
- Before applying L2, scale features.

Lambda [scalar value] is called regularization rate

L2-Regularization

- Goal: Improve generalization performance on unseen data.
- Regularization increases bias (makes model simpler) to reduce variance (sensitivity to noise)

- Implementation
  - Regularization penalty is added to cost function during training. It is *not* used when calculating the final error/metric on the test set.
  - Always scale your data (e.g., StandardScaler) before L2.
  - L2 drives weights toward the origin (zero) but does not set them exactly to zero.
- Hyperparameter λ
  - If λ = 0: Model is standard Linear Regression.
  - Common range: 0.01 to 10 (via Cross-Validation).

L2-Regularization

- How to choose λ?
  - Theory: as ‘n’ grows λ should be in the range O(1) to ($n^{1/2}$).
  - Practice: optimize validation set or cross-validation error.
  - In scikit-learn, λ is called alpha

- Why use L2-Regularization?
  - When properly tuned, it almost always reduces **Test Error**
  - L2 ensures the solution is **unique** and stable.
  - Resulting weights are less sensitive to small "noise"
  - Set λ small to match the performance of a standard model (worst-case scenario).

L1/LASSO Regularization

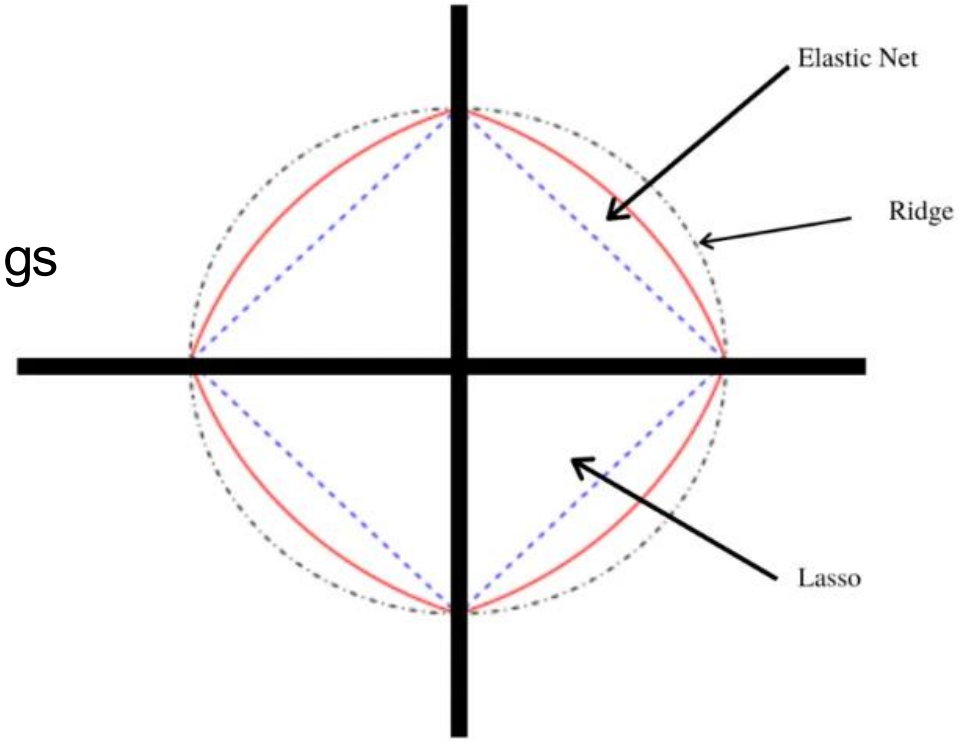
- L2 Regularization (Ridge): Penalizes the square of the weights
  - Effect: Shrinks all weights proportionately toward zero.
- L1 Regularization (Lasso): Penalizes the absolute value of the weights.
  - Effect: Can shrink some weights to exactly zero.

Feature	L2 (Ridge)	L1 (Lasso)
Weight Shrinkage	Weights become very small.	Weights can become zero.
Feature Selection	No (keeps all features).	**Yes** (removes "useless" features).
Solution Stability	Very stable (Unique).	Can be unstable with correlated features.
Computational Efficiency	Analytical solution exists.	Requires coordinate descent or similar.
Best Used When...	Most features are somewhat useful.	You suspect only a few features truly matter.

Elastic Net

- Regularization technique for linear/logistic regression.
- Adds penalty terms to the loss function during training
- Combines L1 (Lasso) and L2 (Ridge) penalties using both absolute and squared coefficient values

- Performs variable selection and regularization simultaneously
- Particularly effective in high-dimensional settings
- Handles groups of correlated predictors more effectively than Lasso alone



Elastic Net

- It controls model complexity, improve generalization, and handle correlated predictors more effectively.
- Regularization strength is controlled by α, while the balance between L1 and L2 penalties is controlled by l1_ratio.
- Elastic Net uses two hyperparameters:
  - α (alpha in scikit-learn) is total penalty strength
    - alpha = 0: No regularization (Linear Regression).
    - alpha = High: Strong regularization (Underfitting).
  - l1_ratio called Mixer: Decides how much of that total penalty is L1 vs. L2.
    - l1_ratio = 1 → Lasso
    - l1_ratio = 0 → Ridge
- The lasso and elastic net are popular regularized regression models for supervised learning.

ElasticNet(alpha=0.1, l1_ratio=0.5) ==> moderate penalty strength that is an equal 50/50 split between Lasso and Ridge.

Elastic Net

- **Elastic Net vs. Lasso and Ridge**
  - Elastic Net addresses limitations of Lasso by including an L2 penalty
  - The L2 component stabilizes coefficient estimates when predictors are correlated
  - When used alone, the L2 penalty corresponds to Ridge regression
  - The combined penalty improves numerical stability and optimization behavior

$$\text{Penalty} = \alpha L1 + (1 - \alpha) L2$$

Summary

- Supervised learning uses labeled data to learn mappings from inputs to known targets
- Two main problem types:
  - Regression: predict continuous values
  - Classification: predict discrete classes
- Key models covered:
  - K-Nearest Neighbors (KNN): instance-based, distance-driven, sensitive to scaling and dimensionality
  - Linear Regression: parametric model minimizing squared error; simple, fast, but sensitive to outliers and nonlinearity
  - Logistic Regression: probabilistic classification model using the logit function; outputs class probabilities

Summary

- Overfitting arises from overly complex models, especially in high-dimensional settings
- Regularization controls model complexity by penalizing large coefficients
  - L2 (Ridge): shrinks coefficients, improves stability, handles multicollinearity
  - L1 (Lasso): promotes sparsity via feature selection
- Elastic Net combines L1 and L2 penalties to:
  - Perform regularization and variable selection simultaneously
  - Handle correlated predictors more effectively than Lasso alone
  - Improve generalization in high-dimensional data
- In practice:
  - α (alpha) controls overall regularization strength.
  - l1_ratio controls the balance between L1 and L2 penalties.